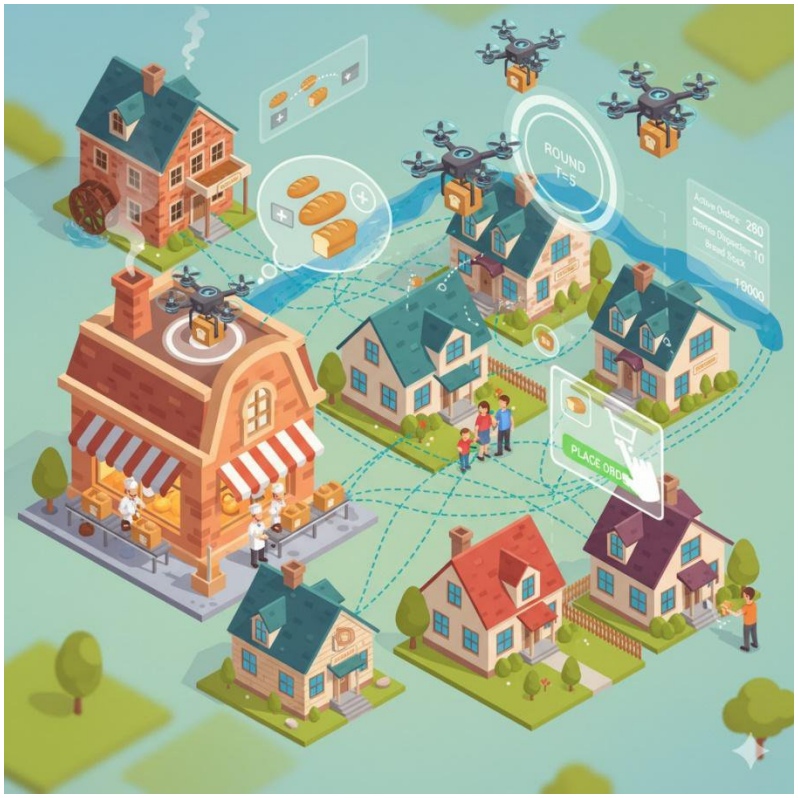

Parallel Bread Delivery Project

1. Project Overview

Objective

Design and implement a **parallel simulation system** for a **bread delivery network** powered by drones.

The system simulates interactions between **bakeries**, **customers**, and a **drone base**. Drones collect bread from bakeries and deliver it to customers according to their orders. The system must schedule drones to fulfill orders by customers in the **most efficient and timely way**, considering drone travel times, locations and customers priority.

The system runs in **discrete rounds**.
At each round:

1. Bakeries produce and restock bread.
2. Customers may place new orders, keep waiting for existing orders, or leave after they receive their bread.
3. The system assigns available drones to orders, optimizing for time and efficiency.
4. Drones move and perform their deliveries.
5. Customers not served in this round increase in priority and wait for the next one.

Learning Goals

- Apply **parallel and distributed programming** concepts to a dynamic simulation.
- Design **data structures** for parallel logistics problems.
- Implement **graph-based routing** and **drone scheduling**.
- Develop heuristics and algorithms for resource allocation and optimization.

2. System Architecture

The system evolves through a series of **rounds** t = 1, 2, ….
Each round includes the following stages:

Initial state

- **Bakeries:**
    - There are several bakeries.
    - Each bakery has:
        - a ***distribution*** of how many breads it creates each round.
          For example:
            - with probability 0.6 it creates 10 breads
            - with probability 0.4 it creates 7 breads
        - a position (x,y)
- **Drone base:**
    - The place where all drones start
    - Has a position (x,y)
- **Customers:**
    - There can be many customers.
    - Each customer has:
        - a position (x,y)
        - an amount of bread they want to order
- **Drones:**
    - There is a limit on how many drones you can create.
    - Each drone has:
        - a capacity (how many breads it can carry)
        - a velocity (how fast it moves)
        - a starting position at the drone base

Stage 1 – State Update

Bakeries: Increase their stock by their production rate.

$$I_b^t = I_b^{t-1} + p_b$$

- t: the round number
- b: the bakery **ID**.
- p_b: the number of breads the bakery **b** produces in the round **t**.
- I_b^t : the total number of breads that available at bakery **b** in round **t**

Customers may:

- Place new orders.
- Leave the system after receiving their orders.
- Wait for the next round if unserved (priority increases).

Drones: Update their position and availability after previous deliveries.

Stage 2 – Order Assignment

Determine which customer orders can be served in this round, based on:

- Available bread inventory.
- Drone positions and travel times.
- Customer priorities.

Orders are selected to maximize efficiency and fairness, balancing **delivery time, bread quantity, and priority weight.**

Stage 3 – Drone Scheduling and Routing

For each customer selected for service:

1. **Bread Collection:**
   Determine from which bakeries the drone(s) will collect bread to meet the customer's order.
2. **Drone Selection:**
   Choose the best drone(s) based on their **current position**.
3. **Route Planning:**
   Plan the drone's route:
   - From its current location
   - To one or more bakeries (for pickup)
   - Then to the customer

4. After pickup:
   The bakeries **subtract** the collected bread from their stock.
5. **Delivery Execution:**
   Drones update their positions and availability times at the end of the round.

Stage 4 – State Transition

After all deliveries:

- **Customers** update their order delivery status.
- **Customers with fulfilled orders** either:
  - Leave the system, or
  - Stay and possibly place new orders in future rounds.
- **Unserved customers** retain their orders and **increase priority**.

3. Core Entities and Data Structures

3.1. Bakery

Represents a bread production site.

Attributes:

- **ID:** unique identifier.
- **Position:** geographic coordinates or a node in a graph.
- **Inventory** I_b**:** current stock of bread.
- **Production rate** p_b**:** number of breads produced each round, drawn from a given distribution
- **Capacity:** maximum possible inventory.

Behavior per round:

- Produces bread according to $\mathbf{p_b}$.
- Supplies drones that collect bread for deliveries.
- If Inventory + $\mathbf{p_b} \geq$ `Capacity, then in this round the bakery produces only Capacity − Inventory breads (it cannot exceed its capacity).`

3.2. Customer

Represents a bread consumer with an order.

Attributes:

- **ID:** unique identifier.
- **Position:** coordinates or graph node.
- **Order quantity** q_c**:** remaining bread demand.
- **Priority** w_c**:** urgency level (increases with waiting rounds).
- **Status**: active, served, or departed.

Priority Update Rule

$$w_c^{t+1} = \{1 \quad if\ a\ new\ order\ is\ placed \mid w_c^t \quad +1 \quad if\ an\ order\ remains\ unserved$$

Behavior per round:

- May create orders.
- Priority increases each round if not served.
- Leaves after receiving all ordered bread (unless reordering).

3.3. Drone Base

A central location where drones are stored initially and may return after deliveries.

Attributes:

- **Position:** fixed base coordinates.
- **Unlimited drones:** new drones can be launched as needed.

Function:

- Serves as the starting point for all drones
- Keeps track of each drone's current position

3.4. Drone

Represents a delivery unit moving between bakeries and customers.

Attributes:

- **ID:** unique identifier.
- **Current position:** where the drone is located (base, bakery, or customer).
- **Velocity v_d :** constant travel speed.
- **Current assignment:** the customer it is currently serving (if any)

Behavior per round:

- Delivers bread according to its assigned route.
- Updates its position and next available time when the delivery is finished.
- Can be assigned to new deliveries in later rounds.

3.5. Graph Representation

Represents the network of all locations (bakeries, customers, base).

Components:

- **Nodes:** bakeries, customers, and the drone base.
- **Edges:** connections (flight paths) between nodes, with a physical distance **d**.
- **Weights:** each edge has:
    - a **distance** d, and
    - a **travel time** t.
      For a drone with velocity $\mathbf{v_d}$, the travel time is
      $t = \frac{d}{v_d}$.
      (Faster drones have smaller t for the same distance.)

Usage:

- Compute shortest paths between any two points.
- Support nearest–neighbour queries and route planning for deliveries.

4. Algorithms Overview

4.1. Assignment Algorithm

Goal: Decide which customers to serve this round and how to allocate bread and drones efficiently.

Inputs:

- Bakery inventories.
- Customer orders and priorities.
- Drone locations and travel times.

Approach - Priority-aware optimization:
For each round select customers that maximizing:

$$\text{Score}(c) = \frac{w_c}{T_c}$$

where T_c is estimated total delivery time (including drone travel and pickup).

4.2. Drone Scheduling Algorithm

Goal: Decide which drone will serve each selected order, taking into account time and location..

Key considerations, For each round:

- Each drone starts from his current position (x,y) or from the base.
- A drone may leave a bakery and, before serving its current customer, return to the same bakery in the next round to pick up more bread for another nearby customer, if this reduces the total delivery time.
- A drone may carry **several orders** on one trip if its capacity allows it. Orders can also be **split between multiple drones** along the route.
- If a drone has no mission, it may move closer to a bakery to be better positioned for future orders.
- **Time computation:**
  For each candidate assignment, compute the **total completion time** using the graph weights:
    - edge distance **d**
    - drone velocity $\mathbf{v_d}$
    - travel time on an edge: $t = \frac{d}{v_d}$

      Sum all travel times along the planned route.
- For each order, choose the drone (or set of drones) with the **minimum total completion time**.

5. Parallelization Design

Parallelizable Components

Component	Description
Bakery updates	Update inventories in parallel
Customer updates	Modify orders, priorities, and arrivals

Component	Description
Drone scheduling	Evaluate drone–order pairs concurrently
Routing	Compute delivery routes for different drones simultaneously
Distance computations	Precompute all pairwise distances in parallel

Synchronization Strategy

- Use a **two-phase model**:
    1. Compute all allocations (orders and drones) without modifying shared states.
    2. Commit inventory and drone updates in a synchronized step.
- This prevents race conditions when multiple threads access shared bakery or drone data.

6. Deliverables

Each student pair must deliver:

1. **Parallel version** of the bread delivery simulation and system design.
2. A **final report** that describes:
    - The system design and architecture
    - The data structures used for bakeries, customers, drones, and the graph
    - The algorithms for assignment, scheduling, and routing
    - The simulation behavior and results for different configurations
    - A discussion of trade-offs, scalability, and realism

The **final report must be submitted as a PDF file**.

7. Suggested Extensions (Optional)

- Introduce **battery limits** or **flight range** for drones.
- Add **delivery time windows** for customers.

8. Evaluation Criteria

Criterion	Weight	Description
Correctness	30%	Accurate simulation logic; state transitions consistent.

Criterion	Weight	Description
Parallelization	60%	Efficient parallel execution with synchronization handled correctly.
Creativity	10%	Added extensions, smart heuristics, visualization.